

KrishiChain V1 — SQL reference

Ten queries, 28 tables, and the demo data behind them. Group A6 · CSE-302 · Project Update-1.

Where: KrishiChainV1/sql/ — run in SQL Developer as **krishichain_demo** / KrishiDemo2026 (localhost:1521, SID xe).

Order: **00_reset** → **01_create_tables** → **02_insert_data** → **03_advanced_queries**.

How: **F5** (Run Script) for the first three. **Ctrl+Enter** for the queries — one at a time, results in a grid.

No indexes, sequences, triggers or PL/SQL anywhere: none of it has been taught yet. Every primary key is written out as a literal value instead of generated.

1 · **ADVANCED QUERIES** — 03_advanced_queries.sql · all ten return rows

Q1 Did the farmer beat the market?

For every completed sale, what the farmer got against that day's published arat price for the same crop, and against the crop's floor price.

five tables · joined on three columns 5 rows

```
SELECT c.CropName           AS crop,
       so.OrderDate         AS sold_on,
       so.AcceptedPricePerKg AS farmer_got,
       dmp.PricePerKg       AS market_price,
       c.BasePrice          AS floor_price,
       CASE WHEN so.AcceptedPricePerKg > dmp.PricePerKg
             THEN 'BEAT MARKET'
             ELSE 'BELOW MARKET'
       END                  AS verdict
FROM   SALE_ORDER so
JOIN   BID b              ON b.BidID   = so.BidID
JOIN   HARVEST_BATCH hb   ON hb.BatchID = b.BatchID
JOIN   CROP c              ON c.CropID  = hb.CropID
JOIN   DAILY_MARKET_PRICE dmp ON dmp.CropID = hb.CropID
                                AND dmp.AratID = hb.AratID
                                AND dmp.PriceDate = TRUNC(so.OrderDate)

ORDER BY so.SaleOrderID;
```

Q2 Which farmers earned the most

Total sales per farmer, ranked.

RANK() OVER (...) over an aggregate 5 rows

```
SELECT u.FirstName || ' ' || u.LastName AS farmer,
       u.District                      AS district,
       COUNT(so.SaleOrderID)            AS orders,
       SUM(so.TotalAmount)              AS total_earned,
       RANK() OVER (ORDER BY SUM(so.TotalAmount) DESC) AS rank_by_earnings
FROM   SALE_ORDER so
JOIN   BID b              ON b.BidID   = so.BidID
JOIN   HARVEST_BATCH hb   ON hb.BatchID = b.BatchID
JOIN   FARM f              ON f.FarmID  = hb.FarmID
JOIN   USERS u             ON u.UserID  = f.FarmerID
GROUP BY u.FirstName, u.LastName, u.District
ORDER BY rank_by_earnings;
```

Q3 Which crops attract the most bidding

Bidding activity per crop, keeping only crops that drew three bids or more.

GROUP BY with HAVING 3 rows

```
SELECT c.CropName           AS crop,
       COUNT(b.BidID)        AS total_bids,
       ROUND(AVG(b.BidPricePerKg), 2) AS average_bid,
       MAX(b.BidPricePerKg)   AS highest_bid
FROM   BID b
JOIN   HARVEST_BATCH hb   ON hb.BatchID = b.BatchID
JOIN   CROP c              ON c.CropID  = hb.CropID
GROUP BY c.CropName
```

```
HAVING COUNT(b.BidID) >= 3
ORDER BY total_bids DESC;
```

Q4 Lots nobody bid on

Produce sitting listed with no bids at all, and whose farm it is on.

NOT EXISTS anti-join

1 row

```
SELECT hb.BatchID           AS batch,
       c.CropName           AS crop,
       u.FirstName || ' ' || u.LastName AS farmer,
       f.FarmName           AS farm,
       f.District           AS district,
       hb.TotalQuantity     AS quantity_kg,
       hb.MinimumPrice      AS asking_price,
       hb.Status            AS status
FROM   HARVEST_BATCH hb
JOIN   CROP c ON c.CropID = hb.CropID
JOIN   FARM f ON f.FarmID = hb.FarmID
JOIN   USERS u ON u.UserID = f.FarmerID
WHERE  NOT EXISTS (SELECT 1
                  FROM BID b
                  WHERE b.BatchID = hb.BatchID)
ORDER BY hb.BatchID;
```

Q5 Is this crop's price rising or falling?

One crop's monthly average price at one arat, with the previous month beside it, so the trend is visible.

LAG() OVER (...) · the previous row

5 rows

```
SELECT c.CropName           AS crop,
       va.AratName          AS arat,
       TO_CHAR(dmp.PriceDate, 'YYYY-MM') AS price_month,
       ROUND(AVG(dmp.PricePerKg), 2)     AS average_price,
       LAG(ROUND(AVG(dmp.PricePerKg), 2)) OVER (
         ORDER BY TO_CHAR(dmp.PriceDate, 'YYYY-MM')) AS previous_month
FROM   DAILY_MARKET_PRICE dmp
JOIN   CROP c ON c.CropID = dmp.CropID
JOIN   VIRTUAL_ARAT va ON va.AratID = dmp.AratID
WHERE  c.CropName = 'Potato'
AND    va.AratID = 1
GROUP BY c.CropName, va.AratName, TO_CHAR(dmp.PriceDate, 'YYYY-MM')
ORDER BY price_month;
```

Q6 Which arat reports to which

RECURSIVE 1

Which arat reports to which, by joining the table to itself: the standard way to read a foreign key that points back at its own table.

self-join · two LEFT JOINS

5 rows

```
SELECT child.AratName       AS arat,
       child.District       AS district,
       parent.AratName      AS reports_to,
       COUNT(hb.BatchID)    AS batches_listed,
       NVL(SUM(hb.TotalQuantity), 0) AS volume_kg
FROM   VIRTUAL_ARAT child
LEFT JOIN VIRTUAL_ARAT parent ON parent.AratID = child.ParentAratID
LEFT JOIN HARVEST_BATCH hb ON hb.AratID = child.AratID
GROUP BY child.AratName, child.District, parent.AratName
ORDER BY parent.AratName NULLS FIRST, child.AratName;
```

Q7 A bidding war, replayed

RECURSIVE 2

A bidding war replayed round by round. The same kind of self-reference as Q6, but followed the whole way down with CONNECT BY rather than one level.

CONNECT BY · START WITH · LEVEL · NOCYCLE

15 rows

```
SELECT b.BatchID           AS batch,
       c.CropName           AS crop,
       LEVEL               AS bid_round,
       u.FirstName || ' ' || u.LastName AS bidder,
       b.BidPricePerKg      AS price_per_kg,
       b.RequestedQuantity  AS quantity_kg,
       b.Status            AS status
```

```

FROM BID b
JOIN USERS u ON u.UserID = b.BuyerID
JOIN HARVEST_BATCH hb ON hb.BatchID = b.BatchID
JOIN CROP c ON c.CropID = hb.CropID
START WITH b.PreviousBidID IS NULL
CONNECT BY NOCYCLE PRIOR b.BidID = b.PreviousBidID
ORDER BY batch, bid_round;

```

Q8 The platform against the physical market

WEAK ENTITY 2

What a crop fetched at a real bazar on a given day, beside the platform's own published price for the same crop that day.

join into a weak entity, through its owner

5 rows

```

SELECT c.CropName AS crop,
       pb.BazarName AS bazar,
       pb.District AS district,
       bdr.RecordDate AS record_date,
       bdr.PricePerKg AS bazar_price,
       dmp.PricePerKg AS platform_price,
       bdr.Revenue AS bazar_day_revenue
FROM BAZAR_DAILY_RECORD bdr
JOIN PHYSICAL_BAZAR pb ON pb.BazarID = bdr.BazarID
JOIN CROP c ON c.CropID = bdr.CropID
JOIN DAILY_MARKET_PRICE dmp ON dmp.CropID = bdr.CropID
                        AND dmp.PriceDate = bdr.RecordDate
ORDER BY c.CropName;

```

Q9 Who is storing what, and has the fee been paid?

TERNARY 1 + WEAK ENTITY 1 + AGGREGATION

Who is storing what, in which unit, on whose authority, and whether the fee has been paid.

six-table join · composite key · LEFT JOIN

7 rows

```

SELECT s.AllocationID AS alloc,
       w.WarehouseName AS warehouse,
       s.UnitNo AS unit,
       su.Capacity AS unit_capacity,
       c.CropName AS crop,
       u.FirstName || ' ' || u.LastName AS authorised_by,
       s.QuantityStored AS stored_kg,
       s.StorageFee AS fee_owed,
       sp.Amount AS fee_paid
FROM STORES s
JOIN WAREHOUSE w ON w.WarehouseID = s.WarehouseID
JOIN STORAGE_UNIT su ON su.WarehouseID = s.WarehouseID
                        AND su.UnitNo = s.UnitNo
JOIN USERS u ON u.UserID = s.ManagerID
JOIN HARVEST_BATCH hb ON hb.BatchID = s.BatchID
JOIN CROP c ON c.CropID = hb.CropID
LEFT JOIN STORAGE_PAYMENT sp ON sp.AllocationID = s.AllocationID
ORDER BY s.AllocationID;

```

Q10 Who drove what

TERNARY 2 + SPECIALIZATION

Which driver took which load in which vehicle, and whether the vehicle was big enough for it.

six-table join across a ternary

5 rows

```

SELECT tr.TransportID AS trip,
       v.VehicleNo AS vehicle,
       v.VehicleType AS vehicle_type,
       v.Capacity AS vehicle_capacity,
       so.AcceptedQuantity AS load_kg,
       u.FirstName || ' ' || u.LastName AS driver,
       tp.LicenseNo AS licence,
       tr.DeliveryStatus AS status
FROM ASSIGNED_TO a
JOIN TRANSPORT_REQUEST tr ON tr.TransportID = a.TransportID
JOIN VEHICLE v ON v.VehicleID = a.VehicleID
JOIN TRANSPORT_PERSONNEL tp ON tp.PersonnelID = a.PersonnelID
JOIN USERS u ON u.UserID = tp.PersonnelID
JOIN SALE_ORDER so ON so.SaleOrderID = tr.SaleOrderID
ORDER BY tr.TransportID;

```

TABLE	PRIMARY KEY	REFERENCES	COLS	ROWS
ADMIN_STAFF subclass of USERS	AdminID	USERS	3	5
ASSIGNED_TO ternary 2	AssignmentID	TRANSPORT_REQUEST, VEHICLE, TRANSPORT_PERSONNEL	6	5
BAZAR_DAILY_RECORD weak entity 2 · cascade	BazarID, RecordDate, CropID	PHYSICAL_BAZAR, CROP, ADMIN_STAFF	7	21
BID recursive 2 (outbid chain)	BidID	HARVEST_BATCH, BUYER, BID	8	15
BUYER subclass of USERS	BuyerID	USERS	4	5
COMPLAINT	ComplaintID	SALE_ORDER, ADMIN_STAFF	7	5
CROP	CropID	CROP_CATEGORY	7	5
CROP_CATEGORY	CategoryID	—	3	5
DAILY_MARKET_PRICE	CropID, AratID, PriceDate	CROP, VIRTUAL_ARAT	6	30
FARM	FarmID	FARMER	9	5
FARMER subclass of USERS	FarmerID	USERS	5	5
HARVEST_BATCH	BatchID	FARM, CROP, VIRTUAL_ARAT	16	8
NOTIFICATION	NotificationID	USERS	9	5
PAYMENT	PaymentID	SALE_ORDER, BUYER, FARMER	9	5
PHYSICAL_BAZAR	BazarID	—	5	5
REVIEW	ReviewID	SALE_ORDER	5	5
SALE_ORDER aggregation · UNIQUE(BidID)	SaleOrderID	BID	9	5
STORAGE_MANAGER subclass of USERS	ManagerID	USERS	2	5
STORAGE_PAYMENT aggregation	StoragePaymentID	STORES	7	5
STORAGE_UNIT weak entity 1 · cascade	WarehouseID, UnitNo	WAREHOUSE	4	11
STORES ternary 1	AllocationID	HARVEST_BATCH, STORAGE_UNIT, STORAGE_MANAGER, FARMER, BUYER, SALE_ORDER	20	7
TRANSPORT_PERSONNEL subclass of USERS	PersonnelID	USERS	3	5
TRANSPORT_REQUEST	TransportID	SALE_ORDER	7	5
USERS	UserID	—	17	25
USER_PHONE	UserID, PhoneNo	USERS	2	29
VEHICLE	VehicleID	—	5	5
VIRTUAL_ARAT recursive 1 (arat tree)	AratID	VIRTUAL_ARAT	7	5
WAREHOUSE	WarehouseID	STORAGE_MANAGER	7	5

The five graded constructs, and where each one lives:

- **Specialization** — USERS into five subclasses, total and disjoint. Each subclass primary key is *also* a foreign key to USERS, so a farmer who is not a user cannot exist. Q10 proves it.
- **Aggregation** — SALE_ORDER hangs off the whole (buyer places bid on batch) fact, via UNIQUE(BidID). STORAGE_PAYMENT does the same for STORES: a fee is owed for the allocation as one thing, not for the batch or the unit or the manager.
- **Two ternaries** — STORES (batch × unit × manager) and ASSIGNED_TO (request × vehicle × personnel). One table each with three FK sets. Splitting them would lose who authorised what.
- **Two weak entities** — STORAGE_UNIT (partial key UnitNo, identified by WAREHOUSE) and BAZAR_DAILY_RECORD (RecordDate, identified by PHYSICAL_BAZAR). Both cascade from their owner.
- **Two recursive** — VIRTUAL_ARAT.ParentAratID (the arat tree) and BID.PreviousBidID (the outbid chain). Q5 and Q7 walk them.

3 · DEMO DATA — 02_insert_data.sql · every table has at least 5 rows

Five farmers in Bogura, Rangpur, Munshiganj, Pabna and Faridpur list eight batches through a three-level arat hierarchy. Batches 1–5 finished bidding — each has an outbid chain ending in a won bid, which became a sale order, which was transported and mostly paid. Batches 6–7 are still open. Batch 8 has no bids at all, because Q3’s anti-join needs one. **The rows are one connected story on purpose:** data generated independently would make these queries return nothing, which is the usual way a demo like this fails.

Tables above five rows, and why:

TABLE	ROWS	REASON
USERS	25	total specialization: 5 per subclass
USER_PHONE	29	some users hold two numbers
DAILY_MARKET_PRICE	30	five months per crop, so Q4 can measure a trend
BAZAR_DAILY_RECORD	21	several crops per bazar per day — the point of the 3-column key
HARVEST_BATCH	8	5 finished auctions + 2 live + 1 with no bids (Q3 needs it)
BID	15	each finished batch carries an outbid chain
STORAGE_UNIT	11	several units per warehouse
STORES	7	leg-1 and leg-2 allocations

Three rules nothing in the database enforces. Each compares two tables, so each would need a trigger — and there are none here. The seed satisfies all three by hand, so check them before editing any price or quantity:

- **BR-09** — a batch’s MinimumPrice must be at least its crop’s BasePrice.
- **BR-11** — a bid must clear the batch minimum and beat the standing highest.
- **BR-18** — a vehicle’s capacity must cover the load it is assigned.

Everything checkable inside one table *is* enforced, by CHECK constraints. The verification block at the end of 02_insert_data.sql re-checks BR-09 and BR-19 and prints a verdict per row.

Two differences from the older database/ scripts, both bringing the tables in line with the Schema Diagram. BAZAR_DAILY_RECORD is now keyed on (BazarID, RecordDate, CropID) with PricePerKg and LoggedBy — the old two-column key allowed only one crop per bazar per day, which defeated the point of the entity. DAILY_MARKET_PRICE has lost LoggedBy: the arat price is computed by the platform, not typed in by an admin.